# FOCEX agent

**User Manual**

**Force constants and phonons from a crystal structure: VASP · SC-SNAPS · FOCEX · Diagnosis & repair**

Keivan Esfarjani

v1.10 · November 2026

https://github.com/KeivanS/Anharmonic-lattice-dynamics (FOCEX/AGENT)

## Contents

**1 What the FOCEX agent does** **2**

**2 Installation, step by step** **2**
2.1 What you need . . . 3
2.2 When step 4 reports a problem . . . 6
2.3 Settings outside the instruction file . . . 6

**3 Quick start** **7**

**4 Mode from_scratch: from a POSCAR to phonons** **9**
4.1 Defaults . . . 11
4.2 The supercell . . . 11
4.3 The FOCEX frame . . . 11
4.4 The k-mesh of the supercells . . . 12

**5 Mode extract_only: FOCEX on existing data** **12**

**6 The instruction file** **12**
6.1 The minimal file . . . 13
6.2 Everything else: the defaults, and how to change them . . . 13

**7 Changing the files the agent writes** **15**

**8 Born charges and the dielectric constant** **17**

**9 The project folder and the results** **17**

**10 When a run stops: diagnose.py** **19**

**11 Thermal conductivity** **19**

**12 Version history** **20**

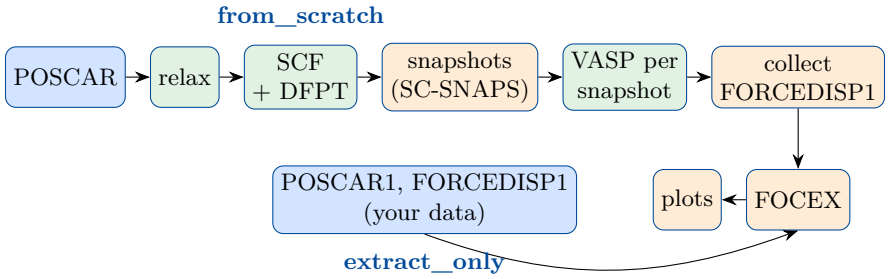
## 1 What the FOCEX agent does

FOCEX obtains the force constants of a crystal (harmonic, cubic, quartic, ...) by fitting them to the forces that density-functional theory (DFT) gives on the atoms of a supercell in which every atom has been displaced a little at random (a *snapshot*). From the force constants it computes the phonon dispersion, density of states, group velocities, Grüneisen parameters, elastic constants, sound speeds and the thermal expansion; THERMACOND then computes the lattice thermal conductivity from the same files.

Making those forces takes many steps: relax the crystal, orient the cell the way FOCEX expects, compute the Born charges of an insulator, choose a supercell, generate the snapshots, run one DFT calculation per snapshot, collect the forces, write the five input files of FOCEX and run it. Each step has its own input files and its own ways to fail. The FOCEX agent, `focex-agent.py`, does them all from one short *instruction file*, checks the result of every step, writes what it did to `report.log`, and can be restarted at any time: finished steps are not repeated. When a run stops, `diagnose.py` finds out where and why, and writes a script that repairs it (Section 10).

It works in one of two modes:

| Mode | You give | The agent does |
|---|---|---|
| **from_scratch** | a crystal structure (`POSCAR`) and the instruction file | VASP relaxation, SCF and Born charges; SC-SNAPS snapshots; one VASP run per snapshot; FOCEX; plots (Section 4) |
| **extract_only** | existing supercells and their forces (`POSCAR1`, `FORCEDISP1`, ...) and the instruction file | rotates the data into the FOCEX frame; FOCEX; plots (Section 5) |




green: VASP; orange: SC-SNAPS, FOCEX and their tools; blue: your input

## 2 Installation, step by step

Do the six steps below in order, once on every computer: a Mac, a Linux workstation, or the login node of a cluster. Only step 1 differs between them. All the settings are made by *one* command, `./configure -install` (step 4): you do not edit `~/.bashrc`, a profile or any other file by hand, and nothing has to be activated afterwards.

**Explanation: how to read this manual**

**Blue boxes** hold what you type, one command per line. Everything after a `#` is a comment and need not be typed. **Amber boxes** like this one explain the step: what it does, and what to do if it fails. They are not commands.

## 2.1 What you need

| What | Needed for | Step |
|---|---|---|
| gfortran, MPI, gnuplot, git | building the codes; running VASP in parallel; the plots | 1 |
| FOCEX, SC-SNAPS, VASP-Flow | the fit, the snapshots, the VASP inputs | 2 |
| Python $\geq$ 3.8 with numpy, spglib, pymatgen | the agent and `focex-diagnose` | 3 |
| VASP 5.4 or newer and its POTCAR library | every DFT step (from_scratch only) | your licence; found in step 4 |

### Step 1. The system tools

**Do this — macOS (with Homebrew, https://brew.sh)**

```
xcode-select --install                # git and the command-line tools
brew install gcc open-mpi gnuplot     # gfortran, mpif90 and mpirun, gnuplot
```

**Do this — Linux (Debian, Ubuntu)**

```
sudo apt install git gfortran libopenmpi-dev openmpi-bin gnuplot python3-venv
```

**Do this — a cluster**

```
module spider gcc                     # the compiler versions (module spider
    openmpi, python: the same)
module load gcc openmpi python        # in this terminal only: step 4 saves them
    for every shell and job
```

**Explanation**

On a cluster, load a gcc, its openmpi and its python *before* steps 3 and 4 (with versions if the cluster wants them, e.g. `module load gcc/14.2.0 openmpi/5.0.7 python/3.13.11`). Do not put `module load` lines into `~/.bashrc` yourself: step 4 writes the right ones into `~/.aladyn.env`, which `~/.bashrc` reads. gnuplot and the VASP module are found by step 4.

### Step 2. Get the codes

**Do this**

```
cd ~
git clone https://github.com/KeivanS/Anharmonic-lattice-dynamics.git
git clone https://github.com/KeivanS/VASP-Flow.git
```

**Explanation**

FOCEX, SC-SNAPS, THERMACOND and the agent are in the first repository; the agent uses VASP-Flow as a library to write the VASP input files and batch scripts (you do not need to learn VASP-Flow). Both are public: no login is needed.

### Step 3. The Python environment

**Do this**

```
python3 -m venv ~/myenv
~/myenv/bin/pip install numpy spglib pymatgen matplotlib flask
```

**Explanation**

One environment, `~/myenv`, serves both VASP-Flow and the agent. It is made once; step 4 finds it (another place: `./configure -install -python /path/to/bin/python`). On a cluster it can only start with the module of the Python it was made from: step 4 finds that module from `~/myenv/pyvenv.cfg` and loads it in every shell and job.

### Step 4. Configure: one command

**Do this**

```
cd ~/Anharmonic-lattice-dynamics
./configure --install                  # ends with "Ready"; on a cluster add:
    --account <SLURM account>
```

**Explanation**

It finds what this computer has, tests it, writes all the settings and checks them in a new login shell. Every item is reported as **ok**, **WARN** (optional, or needed only for some calculations) or **FAIL** (needed); for each problem the end of the report gives the command that fixes it. Fix it and run the same command again: running it again is always safe.

**What it looks at:** the processor and cores, SLURM, the modules, gfortran and mpif90 (it compiles and runs an OpenMP and an MPI test program), `~/myenv` and its packages, VASP-Flow, the POTCAR library (`VASP_POTCAR_DIR`, else folders named `potpaw_PBE*` or `PAW_PBE*` in your home folder, `/opt`, `/usr/local`, `/share`, `/apps`, `/sw`), `vasp_std` (a binary whose libraries are missing, `ldd ... not found`, is rejected), gnuplot (with the `pdfcairo` terminal the plots use; on a cluster the `gnuplot` module, if it has it).

**What it writes** (the previous version of each is kept as `.bak`):

- `make.local`: compilers, `DESTDIR` (`~/ALADYN/bin`) and `PYTHON` for the Makefiles;
- `~/.aladyn.env`: the `module load` line (cluster), `~/ALADYN/bin` in `PATH`, `FOCEX_PROFILE` (cluster), and it reads `~/.focex-agent.env`;
- `~/.focex-agent.env`: `VASP_POTCAR_DIR`, `VASP_STD` (workstation), `MPI_LAUNCH`, `VASP_FLOW_DIR`, and `~/myenv/bin` at the front of `PATH`;
- one line at the end of `~/.bashrc` (Linux), `~/.bash_profile` (Mac with bash) or `~/.zshrc` (zsh) that reads `~/.aladyn.env`; never twice;
- the launchers `focex-agent`, `focex-diagnose`, `aladyn-gui`, `allplots`, `kappaplots` in `~/ALADYN/bin`: they start the scripts with `~/myenv`'s Python, so they work in any shell and batch job;
- on a cluster: the SLURM profile and `~/ALADYN/bin/vasp-env` (box below), and THERMACOND job examples in `~/ALADYN/jobs`.

**If it finds the wrong thing, or nothing**, tell it: `--potcar /path/to/potpaw_PBE.54` (the folder with `Si/`, `O/`, ...), `--vasp /path/to/vasp_std`, `--python /path/to/python`, `--account NAME`, `--vasp-module NAME`.

**Explanation: on a cluster, the profile**

On a cluster VASP is started by SLURM, so the agent needs a *profile*: a small JSON file in `~/VASP-Flow/profiles/`. Step 4 drafts it as `<cluster name>.json` (`--profile-name` for another name) and names it in `~/.aladyn.env` as `FOCEX_PROFILE`, the default of the agent and of `aladyn-gui`. In it: the default partition and its cores per node (`sinfo`), 1 node, 12 hours, the modules of step 1, and the account. **The account**: `--account NAME`; without it the one of the profile drafted before, else your SLURM default account; with several accounts a WARN lists them all. **VASP**: the newest `vasp/<version>` module (`--vasp-module` for another), tested: it must give a `vasp_std` whose libraries are all found.

On some clusters the VASP module loads another compiler (e.g. Intel) and so unloads gcc and the Python module, which the agent needs in the same job. Step 4 tests that; if it happens, the job keeps the modules of step 1 and VASP is started as `~/ALADYN/bin/vasp-env srun vasp_std`: `vasp-env` loads the VASP module for the VASP command only (`mpi_cmd` of the profile). Otherwise the VASP module is simply one of the profile's modules and `mpi_cmd` is `srun`.

A profile drafted by step 4 is drafted again when step 4 is run again (the old one is kept as `.bak`); a profile you wrote yourself (without the "Drafted by ALADYN's configure" comment) is never changed. To check it: `cat ~/VASP-Flow/profiles/$FOCEX_PROFILE.json`.

**The number of VASP ranks.** With `srun` VASP runs on all tasks of the job, nodes × `ntasks_per_node` (`NODES:`, `NTASKS_PER_NODE:`, `WALLTIME:`, `PARTITION:` in the instruction file replace those of the profile). Without an `MPI` line that is the number of ranks, and KPAR and NCORE are chosen for it. `MPI: N` fewer than the tasks is run as `srun -n N` (a WARNING says the others stay idle); more than the tasks stops the set-up. On a workstation no profile is needed: VASP is started with `MPI_LAUNCH` on `MPI` ranks, and `run_workflow.sh` with `bash`.

## Step 5. Build

**Do this**

```
make                                  # in ~/Anharmonic-lattice-dynamics; a few
    minutes
```

**Explanation**

Builds FOCEX, SC-SNAPS, `read_outcar.x` and the other utilities, THERMACOND (MPI and serial) and ANFOMOD with the compilers of `make.local`, into `~/ALADYN/bin`. (`./configure -install -build` does steps 4 and 5 in one go.)

## Step 6. Check, in a new terminal

**Do this**

```
focex-agent --version                 # prints: focex-agent 1.10
echo $VASP_POTCAR_DIR $FOCEX_PROFILE  # the POTCAR library; on a cluster, the
    profile's name
```

**Explanation**

Step 4 has already made this check in a fresh login shell (its section "8. Check in a new login shell"); here you see it yourself. Nothing needs to be activated: `focex-agent` starts with `~/myenv`'s Python by itself. If `focex-agent: command not found`: the start-up file was not read; on a Mac check that the terminal runs the shell `echo $SHELL` names, then run step 4 again.

### Updating

**Do this**

```
cd ~/Anharmonic-lattice-dynamics && git pull && ./configure --install && make
cd ~/VASP-Flow && git pull
```

## 2.2 When step 4 reports a problem

| Message | What to do |
|---|---|
| `gfortran 8.5: version 9 or newer is needed` | cluster: `module load gcc openmpi python` (step 1), then step 4 again; Mac: `brew install gcc` |
| `~/myenv/bin/python does not start` | the module of its Python was not found: load it (`module spider python`), or make `~/myenv` again with the Python loaded now (step 3) |
| `no Python with numpy, spglib and pymatgen` | step 3 |
| `vasp_std cannot run here: N of its libraries are missing` | an old VASP build (e.g. after a system update): on a cluster nothing to do, the VASP module is used; on a workstation rebuild VASP or give `--vasp` |
| `account X is not one of yours` | `./configure -install -account <one of the names listed>` |
| `several SLURM accounts ...: X is used` | if X is the wrong one: `--account NAME` |
| `module ... gives no vasp_std / misses libraries` | `module spider vasp`, then `--vasp-module NAME` |
| `no POTCAR library found` | `--potcar /path/to/potpaw_PBE.54` |
| `the module gnuplot/... has no pdfcairo terminal` | nothing to do: the agent writes the plot scripts; run `makeplots.sh` (in `05_focex`) on a computer whose gnuplot makes PDFs |
| `VASP-Flow not found` | step 2 |
| `VASP 5.4.4 has no r2SCAN` | r2SCAN, the default `FUNCTIONAL`, needs VASP 6: VASP 5 reads `METAGGA = R2SCAN` without a word and runs PBE. Put `FUNCTIONAL: PBE` (or `PBEsol`) in the instruction file, or use a VASP 6 module (`--vasp-module`). The agent warns at the set-up, and the relaxation stops at its first OUTCAR if `METAGGA` was ignored |
| `focex-agent does not start in a new shell` | read the lines above it; usually one of the problems above |

## 2.3 Settings outside the instruction file

These are environment variables; step 4 sets them for you.

| variable | if not set | what it is |
|---|---|---|
| `VASP_POTCAR_DIR` | no POTCAR is made | the PBE POTCAR library: one folder per element |
| `VASP_POTCAR_DIR_LDA` | PBE folder with PBE→LDA in its name | the LDA library, for `FUNCTIONAL: LDA` |
| `VASP_STD` | `~/BIN/vasp_std` | the VASP executable (workstation; a profile overrides it) |
| `MPI_LAUNCH` | `mpirun -np` | the command before the number of ranks and VASP |
| `VASP_FLOW_DIR` | `~/VASP-Flow` | where VASP-Flow is |
| `FOCEX_PROFILE` | no profile | the VASP-Flow profile used without `PROFILE:` (`./configure` on a cluster) |
| `VASP_OMP_NUM_THREADS` | 1 | OpenMP threads per MPI rank of VASP (box below) |

**Note. Threads.** You do not have to do anything about OpenMP threads: the agent runs VASP with one thread per MPI rank (`OMP_NUM_THREADS=1`). Without that, a BLAS library built with OpenMP starts one thread per core in every MPI rank and the cores are oversubscribed many times (a 96-atom ZnO supercell made no SCF step in 30 minutes on 8 ranks). Only if your VASP is built for hybrid MPI + OpenMP runs and you want them, set `VASP_OMP_NUM_THREADS`.

## 3 Quick start

A from_scratch calculation of silicon on a workstation (Section 5 has extract_only).

### 1. A folder with the crystal structure

**Do this**

```
mkdir -p ~/work/Si && cd ~/work/Si
cp /path/to/your/POSCAR POSCAR
```

**Explanation**

The structure in VASP's POSCAR format, named `POSCAR` (another name: `--coord FILE`). Any cell of the crystal will do – primitive or not, any orientation: the agent reduces and orients it.

### 2. The instruction file

**Do this**

```
cat > focex-agent-instructions.txt <<'EOF'
Project: Si
RANKS: 1 1 1 0 0 0 0 0
MPI: 8
EOF
```

**Explanation**

This is the minimal file; Section 6 explains each line and every keyword that you may add. `MPI` is the number of cores VASP uses: `./configure` prints a suggestion for this computer. With a SLURM profile it may be left out: VASP then runs on the job's tasks, nodes × ntasks_per_node of the profile (step 4, the profile box).

### 3. Set up the project

**Do this**

```
focex-agent                              # on a cluster: the profile of ./configure,
    or --profile NAME
```

**Explanation**

This takes seconds and runs no VASP. It creates the project folder (`Si`, after `Project:`; `PROJECT_DIR:` puts it elsewhere), with the relaxation inputs and `run_workflow.sh`, and prints the command that starts it. If the folder holds a project already, it stops; `--force` starts over. Before anything runs you can read, e.g., `Si/01_relax/INCAR`.

### 4. Start the workflow and follow it

**Do this**

```
cd Si
nohup bash run_workflow.sh > workflow.out 2>&1 &    # on a cluster: sbatch
    run_workflow.sh
tail -f report.log                                   # Ctrl-C stops the display,
    not the run
```

**Explanation**

`report.log` has one entry per task: OK or FAILED, the time, and what was checked (Section 4). Starting `run_workflow.sh` again continues where it stopped; finished tasks are not repeated.

### 5. If it stops

**Do this**

```
focex-diagnose ~/work/Si/Si
less ~/work/Si/Si/diagnosis/report.txt
bash ~/work/Si/Si/diagnosis/fix_it.sh
```

**Explanation**

`focex-diagnose` (`diagnose.py`) changes nothing itself; it writes what failed and why, and a script that repairs it (Section 10).

The options of `focex-agent` are all optional:

| option | meaning |
|---|---|
| `--instr FILE` | the instruction file, if it is not called `focex-agent-instructions.txt` |
| `--coord FILE` | the crystal structure, if it is not called `POSCAR` (from_scratch) |
| `--profile NAME` | the VASP-Flow profile; default: `PROFILE:` of the instruction file, else `FOCEX_PROFILE` (step 4 sets it on a cluster), else none. Kept for the whole run |
| `--force` | set up again a project folder that exists already. Its record of finished tasks is reset, so use it only to start over, after removing the old run folders |

## 4 Mode from_scratch: from a POSCAR to phonons

`run_workflow.sh` runs `focex-agent.py run <PROJECT_DIR>`, which does the eleven tasks below in order. Each finished task is recorded in `state.json` and in `report.log`, with what was checked; a task that fails stops the workflow, and starting `run_workflow.sh` again continues from that task. A job that hits its time limit is simply submitted again.

The supercell tasks are checked again before they are skipped:

- **A removed or incomplete `04_supercells/snap_NNN` folder** is written again, and the `supercells` task and every task after it are redone. Only the folders without a converged OUTCAR run VASP.
- **To leave snapshots out on purpose**, move their folders to `04_supercells/unused/`. They are then not recreated, and `FORCEDISP1` is collected again from the others.
- **A `snap_NNN` folder with no `03_snaps/poscar_NNN`** stops the workflow.
- **To add snapshots**, raise SNAPSHOTS in the project's `focex-agent-instructions.txt` and start `run_workflow.sh` again. The extra snapshots are made at the same temperature and numbered after the existing ones, which are kept; only the new ones run VASP, then FOCEX is run again on all of them. A lower SNAPSHOTS removes nothing.

**relax** The structure you gave is reduced to its primitive cell (Niggli reduction) and rotated into the orientation FOCEX and SC-SNAPS use (Section 4.3); the result is `POSCAR.init`. Initial moments given atom by atom (a `MAGMOM:` list or `MAGMOM =` in an INCAR block, one value per atom, or three for non-collinear moments) keep the cell you gave: atoms with different moments count as different, so an antiferromagnet keeps its magnetic cell, and the report says when that order would also fit a smaller cell. One value per element (`MAGMOM: Fe=4`) does not stop the reduction. VASP relaxes the atoms and the cell shape and volume in `01_relax`. If the forces are not yet below EDIFFG after NSW ionic steps, the relaxation continues from its last structure (CONTCAR), up to three times.

**poscar0** The relaxed cell is reduced and oriented again and written as `POSCAR0`: every later step uses this cell and this Cartesian frame. The report gives its space group and lattice parameters.

**scf** A self-consistent calculation on POSCAR0 in `02_scf` (same INCAR, no ionic steps, same k-mesh), which also gives the band gap. If the cell did not change in the previous task, it starts from the charge density and wave functions of the relaxation.

**dfpt** For an insulator (band gap above 0.05 eV, `DFPT: auto`): density-functional perturbation theory in `06_dfpt` (as in VASP-Flow; `02_scf/dfpt` in projects made before 2026-11-07) gives the high-frequency dielectric constant and the Born effective charges of every atom

(Section 8). `DFPT: on` forces it, `off` skips it. With the finite-field method (`LCALCEPS`) the field `EFIELD_PEAD` is set to half of VASP's limit for Zener tunnelling, $e|E|\,|a_i| < E_g/(10N_i)$ ($E_g$ the SCF gap, $N_i$ the k-points along $\mathbf{a}_i$; at most VASP's default 0.01 eV/Å), unless you set it in `INCAR_dfpt`. The values are read from the `OUTCAR`; `extract_born.py` also writes them to `born_charges.txt` and `BORN` (its output in `extract_born.out`). The result is checked: no Zener warning from VASP, Born charges summing to zero (within 0.1 e per component), $\varepsilon_\infty$ between 1 and 100. After a Zener warning the run is repeated once with the smaller field. If a check still fails, `report.log` gives a WARNING and the charges are not used: FOCEX runs with `born_flag` 0, unless you give them in a `dielectric_params` block.

**`snaps`** The supercell is chosen (Section 4.2). SC-SNAPS (`03_snaps`) then writes the undistorted supercell `poscar_000` and 50 snapshots, each atom displaced as in a thermal ensemble of harmonic oscillators of frequency `AVG_FREQ` at temperature `TEMPERATURE`. **A temperature you give is used as it is**, e.g. to sample the displacements of that temperature for effective phonons; the report gives the resulting rms displacement. Only when `TEMPERATURE` is not given, the default 30 K is adjusted (SC-SNAPS rerun at most twice) until the rms displacement is within 15 % of `DISP_TARGET` (0.02 Å): large enough for clean forces, small enough for the Taylor expansion of the energy. The undistorted supercell is checked to be an exact multiple of POSCAR0 before any VASP job is run.

**`supercells`** One folder per snapshot, `04_supercells/snap_NNN`, with its POSCAR, the SCF INCAR without symmetry (`ISYM = 0`) and a k-mesh equivalent to the SCF one (Section 4.4); magnetic moments given per atom are carried over to the supercell.

**`vasp_snaps`** The VASP runs of the snapshots, one after the other. A run that fails does not stop the others; the failed ones are listed in `report.log`, and `resubmit_failed.sh` reruns only those. An existing OUTCAR is never overwritten (it is kept as `OUTCAR.1`, ...).

**`collect`** `read_outcar.x` reads the positions and forces of every run into `04_supercells/FORCEDISP1`; `POSCAR1` is the undistorted supercell. The report gives the largest net force on a supercell (should be $\approx 0$) and the largest difference between the positions in OUTCAR and in POSCAR. The undistorted supercell (`snap_000`, run with `ISIF = 2`; the other snapshots use `ISIF = 1`, which gives only the trace) also gives the stress of the reference structure: `stress.params` (GPa, tensile positive, i.e. minus the `in kB` line of OUTCAR divided by 10) is written and copied to the FOCEX folders, where it enters the Huang rows and the elastic constants.

**`focex`** The five input files of FOCEX are written in `05_focex` (`structure.params`, `latdyn.params`, `kpbs.params`, `dielectric.params`, `default.params`) and FOCEX is run with the invariances held exactly (`enforce_inv = 1`). With Born charges a second run without them is made in `05_focex/NOBORN`, for comparison; with fewer than `LASSO_RATIO` (2) equations per unknown, a third run with LASSO and a refit (`enforce_inv = 3`), which keeps only the force constants the data need, is made in `05_focex/LASSO`. The report gives the fit error, the numbers of equations and unknowns, the lowest $\omega^2$ (a warning below $-25\,\mathrm{cm}^{-2}$: imaginary modes) and the highest frequency. If the DOS range of `latdyn.params` does not fit the frequencies, FOCEX is run again with one that does.

**`plots`** `FOCEX/PLOTTING/allplots` fills the gnuplot scripts (bands, band Gruneisen parameters, group velocities, DOS, QHA, FCs) and makes the PDFs in every FOCEX folder (Section 9); `makeplots.sh` runs it again.

`thermacond` THERMACOND in `07_thermacond`: linewidths or the thermal conductivity, when `THERMACOND_JOB` is 1, 2 or 3; skipped for 0, the default (Section 11).

## 4.1 Defaults

- `ACCURACY: Accurate`: ENCUT 600 eV, EDIFF $10^{-7}$ eV, EDIFFG $-0.003$ eV/Å, PREC Accurate in every VASP step; the relaxation uses IBRION = 1 (quasi-Newton). `Normal`: 450 eV, $10^{-6}$, $-0.003$, PREC Normal – cheaper, but the residual forces of the relaxed structure are larger (0.01 eV/Å in a ZnO test), which adds noise to the force constants. DFPT uses EDIFF $10^{-8}$.
- Relaxation and SCF k-mesh: VASP-Flow's `coarse` density (1000 k-points per reciprocal atom), `fine` (6000) if every element is a metal. Each subdivision is rounded to the nearest integer; VASP-Flow itself rounds to even numbers (multiples of 6 in plane for hexagonal cells), the agent does not. The FOCEX mesh (`FOCEX_MESH`) is made the same way.
- 50 snapshots. Each gives three equations per atom of the supercell: 20 snapshots of a 96-atom supercell give 5760 equations for the 264 unknowns of ZnO with ranks 1–3, so fewer snapshots are often enough (`SNAPSHOTS`).
- `RANGE: 6 2 1 0 0 0 0`: neighbour shells kept for ranks 2, 3, 4, ...; `FC2RANGE: 0`: the harmonic force constants are kept over the whole supercell.
- FOCEX k-mesh `fine`; the band-structure path is found automatically.

## 4.2 The supercell

`SUPERCELL:` takes one of three forms:

- `SMALL` (the default): the multiple $m_1 \times m_2 \times m_3$ of the conventional cell with the fewest atoms whose faces are at least 10 Å apart, at most 100 atoms;
- `LARGE`: the same with 14 Å and at most 300 atoms. In both, if no such multiple is small enough, the one with the largest face distance; it keeps the symmetry of the lattice;
- **the supercell matrix itself**: nine integers, the three rows of the matrix that the agent writes to SC-SNAPS's `supercell.inp`. The first three integers $n_{11}\, n_{12}\, n_{13}$ give the first supercell vector as a combination of the primitive vectors of POSCAR0, $\mathbf{R}_{S1} = n_{11}\mathbf{R}_{01} + n_{12}\mathbf{R}_{02} + n_{13}\mathbf{R}_{03}$; the next three give $\mathbf{R}_{S2}$ and the last three $\mathbf{R}_{S3}$ in the same way (the agent's `cell.inp` has `convcoord = 1`, primitive units). For example `SUPERCELL: 2 0 0 0 2 0 0 0 2` is $2 \times 2 \times 2$ primitive cells, and for an fcc crystal `SUPERCELL: -1 1 1 1 -1 1 1 1 -1` is its conventional cube (4 primitive cells). The determinant must be positive.

The report gives the matrix and the number of atoms (for SMALL and LARGE also the face distance).

## 4.3 The FOCEX frame

FOCEX and SC-SNAPS build the conventional cell from $a, b, c, \alpha, \beta, \gamma$ with $a$ along $x$ and $b$ in the $xy$ plane, and the supercells, forces and Born charges must be in that same Cartesian frame. The agent rotates the cell into it at the start, so everything VASP computes is already in the right frame. `structure.params` lists the conventional cell (line 1), the primitive vectors in its units (line 2) and the atoms in conventional coordinates.

### 4.4 The k-mesh of the supercells

If the supercell is a diagonal multiple of the primitive cell, its mesh is the SCF mesh divided by the multiples, rounded up (6 6 4 with a $3\times3\times2$ supercell gives 2 2 2). Otherwise (every multiple of a centred conventional cell) it keeps the k-point spacing of the SCF mesh. `KPOINTS_snaps` sets it yourself (Section 7).

## 5 Mode extract_only: FOCEX on existing data

Use it when the supercell forces exist already – from an earlier calculation, another code, or a colleague – and you want FOCEX's force constants and phonons. No VASP and no SC-SNAPS are run.

**The data folder** (named in the instruction file with `DATA:`) holds

- `POSCAR1` and `FORCEDISP1`: a supercell and the positions and forces of its snapshots, in FOCEX's format (what `read_outcar.x` or `read_qe.x` write); more pairs `POSCAR2`/`FORCEDISP2`, ... are all used. Any orientation.
- optionally `POSCAR`: the primitive cell, in the same orientation as the data; without it, spglib finds the primitive cell of `POSCAR1`.
- optionally `dielectric.params`: the dielectric constant and the Born charges, in the frame of the data, one $3\times3$ block per atom of that primitive cell. With more than one atom of a species the primitive `POSCAR` is required, to fix the order of the atoms.

`SPECIES: Mg O` names the elements when the POSCARs have no element line.

**What the agent does.** At set-up it symmetrizes the lattice of the primitive cell (DFT lattice parameters given to 6–7 digits make FOCEX's symmetry analysis lose force constants), finds the rotation into the FOCEX frame, and rotates the supercells, the positions and forces in `FORCEDISPi`, the dielectric constant and the Born charges with it. The report gives the rotation and the size of the symmetrization. The workflow then has only the tasks focex and plots.

```
cat > focex-agent-instructions.txt <<'EOF'
Project: MgO
MODE: extract_only
DATA: ~/data/MgO_forces          # POSCAR1, FORCEDISP1, dielectric.params
SPECIES: Mg O
RANKS: 1 1 1 0 0 0 0 0
EOF
python ~/Anharmonic-lattice-dynamics/FOCEX/AGENT/focex-agent.py
cd MgO && bash run_workflow.sh          # FOCEX takes minutes: no batch job needed
```

Checked on MgO data given in a rhombohedral orientation: the fit error (7.197 %), the 35 unknowns, the frequencies and the elastic moduli are those of a FOCEX run in the original frame.

## 6 The instruction file

**Explanation: the format**

One keyword per line, `KEYWORD: value`; upper or lower case; `#` starts a comment. Blocks that change whole generated files are described in Section 7. A commented example of every line is `FOCEX/AGENT/focex-agent-instructions.txt`.

## 6.1 The minimal file

**Do this — from_scratch: start from a POSCAR**

```
Project: Si
RANKS: 1 1 1 0 0 0 0 0
MPI: 8
```

**Explanation**

`RANKS` **the only keyword that must be given**; the agent stops without it. Eight flags, one per rank of force constants 1–8: `1 1 0 0 0 0 0 0` harmonic, `1 1 1 0 0 0 0 0` harmonic and cubic (needed for Grüneisen parameters, thermal expansion, thermal conductivity), `1 1 1 1 0 0 0 0` up to quartic.

`Project` a name, used in titles and as the name of the project folder. Without it the name is `VASP_Calculation`.

`MPI` the number of cores (MPI ranks) VASP uses. **Without it VASP runs on one core**, on a workstation; with a SLURM profile it runs on all the job's tasks.

On a cluster nothing needs to be added: step 4 of Section 2 made the profile, with the account (`PROFILE:` and `ACCOUNT:` choose others for one project).

**Do this — extract_only: FOCEX on forces you already have**

```
Project: MgO
DATA: ~/data/MgO_forces
RANKS: 1 1 1 0 0 0 0 0
```

**Explanation**

`DATA` is the folder with `POSCAR1` and `FORCEDISP1` (Section 5); giving it selects `MODE: extract_only`. No VASP is run, so `MPI` is not needed.

## 6.2 Everything else: the defaults, and how to change them

Every keyword below may be left out; the second column is what the agent then does. To change it, add the line of the third column (with your value) to the instruction file.

| keyword | if left out | to change it, e.g. |
|---|---|---|
| **project** | | |
| `PROJECT_DIR` | folder named after Project, next to the instruction file | `PROJECT_DIR: /scratch/me/Si` |
| `MODE` | from_scratch; extract_only when DATA is given | `MODE: extract_only` |
| `SPECIES` | element names from the POSCARs (extract_only) | `SPECIES: Mg O` |
| `PROFILE` | none: a workstation, `bash run_workflow.sh` | `PROFILE: mycluster` |
| `ACCOUNT` | the account of the profile (SLURM) | `ACCOUNT: my_allocation` |

| keyword | if left out | to change it, e.g. |
|---|---|---|
| STOP_AFTER | all tasks run | STOP_AFTER: relax (stops after that task; delete the line and submit run_workflow.sh again to go on) |
| **force constants** | | |
| RANGE | 6 2 1 0 0 0 0: neighbour shells for ranks 2–8 | RANGE: 8 3 2 0 0 0 0 |
| FC2RANGE | 0: harmonic range = the supercell | FC2RANGE: 0 |
| LASSO_RATIO | 2: below 2 equations per unknown an extra LASSO run | LASSO_RATIO: 3 |
| **VASP** | | |
| FUNCTIONAL | R2SCAN | FUNCTIONAL: PBEsol (PBE, PBEsol, LDA, AM05, R2SCAN, HSE06, VV10) |
| ACCURACY | Accurate: ENCUT 600 eV, EDIFF $10^{-7}$, EDIFFG $-0.003$, PREC Accurate | ACCURACY: Normal (450 eV, $10^{-6}$, $-0.003$) |
| ENCUT | from ACCURACY | ENCUT: 520 |
| KMESH_DENSITY | coarse (1000 k-points per reciprocal atom); fine (6000) if every element is a metal | KMESH_DENSITY: fine (or a number) |
| MAGMOM | no magnetic moments | MAGMOM: Cr=3.0 |
| GGA_U | auto: tabulated U on d/f elements if the compound has O, S, Se, Te or a halogen | GGA_U: OFF (or ON) |
| DFPT | auto: Born charges and $\varepsilon_\infty$ if the SCF shows a gap (>0.05 eV) | DFPT: off (or on) |
| BORN_TRANSPOSE | yes (Section 8) | BORN_TRANSPOSE: no |
| STRESS_OFFSET | none | STRESS_OFFSET: -5.54 -5.54 -6.28 (kB, xx yy zz: known error of VASP's printed stress, printed minus $-(1/V)\,dE/d\varepsilon$, e.g. rVV10 in VASP $\leq$ 6.2; removed before stress.params is written; the relaxation needs PSTRESS in INCAR_relax) |
| VASP_RUN | here: the agent starts VASP | VASP_RUN: external (inputs written, the agent stops as WAITING and goes on from the outputs; aladyn-gui bundles or submits them) |
| THERMACOND_RUN | here | THERMACOND_RUN: external (the same for THERMACOND) |
| **snapshots** | | |

| keyword | if left out | to change it, e.g. |
|---|---|---|
| SUPERCELL | SMALL: faces ≥10 Å apart, ≤100 atoms (Section 4.2) | SUPERCELL: LARGE or SUPERCELL: 2 0 0 0 2 0 0 0 2 |
| SNAPSHOTS | 50 | SNAPSHOTS: 20 |
| TEMPERATURE | 30 K, adjusted until the rms displacement is DISP_TARGET | TEMPERATURE: 300 (used as given) |
| DISP_TARGET | 0.02 Å (only without TEMPERATURE) | DISP_TARGET: 0.03 |
| AVG_FREQ | 400 $cm^{-1}$, frequency of the SC-SNAPS oscillators | AVG_FREQ: 250 |
| **FOCEX** | | |
| FOCEX_MESH | fine | FOCEX_MESH: 12 12 12 (or coarse, or a density) |
| KPATH | 0: band path found automatically | KPATH: 1 (and a kpbs_params block) |
| BS_NKPTS | 40 points per segment of the band path | BS_NKPTS: 60 |
| **THERMACOND** (Section 11) | | |
| THERMACOND_JOB | 0: not run | THERMACOND_JOB: 2 (1 linewidths, 2 or 3 $\kappa$) |
| THERMACOND_MESH | the FOCEX mesh (usually far too dense) | THERMACOND_MESH: 12 12 12 |
| THERMACOND_T | 0 900 31: Tmin, Tmax (K), number of temperatures | THERMACOND_T: 100 900 9 |
| THERMACOND_SOLVER | 0: conjugate gradient (job 2) | THERMACOND_SOLVER: 1 (SVD) |
| **programs** | | |
| FOCEX_X | the newest ~/ALADYN/bin/focex-*.x | FOCEX_X: /ALADYN/bin/focex-9.8.x |
| SC_SNAPS_X | ~/ALADYN/bin/sc_snaps.x (else ~/BIN/sc_snaps.x) | SC_SNAPS_X: /path/sc_snaps.x |
| READ_OUTCAR_X | ~/ALADYN/bin/read_outcar.x | READ_OUTCAR_X: /path/read_outcar.x |
| THERMACOND_X | the newest ~/ALADYN/bin/tcond-*.x (MPI build; the serial one if there is no MPI build) | THERMACOND_X: /path/tcond-2.2-serial.x (serial build: one process on the job's cores as threads, no srun) |

Further VASP keywords are those of VASP-Flow (its manual lists them all); single INCAR tags are best given in an INCAR block (Section 7).

## 7 Changing the files the agent writes

Every file the agent generates can be changed *from the instruction file*, with a block named after the file: a line with its name, the contents, and a line with the name followed by _END.

- **INCAR blocks are merged tag by tag**: the tags you give replace or add to those the agent writes; the others stay. INCAR applies to every VASP step; INCAR_relax, INCAR_scf, INCAR_dfpt, INCAR_snaps to one.

- **Magnetic order and +U are the same in every step**: `MAGMOM`, `ISPIN`, `NUPDOWN`, `LNONCOLLINEAR`, `LSORBIT`, `SAXIS`, `M_CONSTR`, `I_CONSTRAINED_M`, `LAMBDA`, `LDAU`, `LDAUTYPE`, `LDAUL`, `LDAUU`, `LDAUJ` and `LMAXMIX`, given in any INCAR block (`INCAR_scf` too), go to the relaxation, the SCF, DFPT and the supercells; the same tag with different values in two blocks stops the agent. A `MAGMOM` (or `M_CONSTR`) with one value, or three for non-collinear moments, per atom of the cell is written for the supercells with each atom's value taken from its site in the cell (an antiferromagnet keeps its order); a list of the supercell's length is used as it is, any other length stops the supercells task. `LDAUL/U/J` are per species, and the supercells have the species of the cell. Give the per-atom list in the order of your POSCAR: the agent keeps that order. `04_supercells/snapshots_mag.xyz` shows the moments of every supercell (Jmol's xyz format, the moment as the vector of each atom, collinear ones along $z$; in Jmol `vectors on`); the POSCARs are left as VASP reads them.
- **Every other block replaces the whole file**, so it must be complete: `KPOINTS` (relaxation and SCF), `KPOINTS_snaps`, `cell_inp`, `supercell_inp`, `snaps_inp` (SCSNAPS), `structure_params`, `latdyn_params`, `kpbs_params`, `dielectric_params`, `default_params` (FOCEX).

Simple choices – the supercell, the number of snapshots, the FOCEX k-mesh – have their own keyword; use a block only for what has none.

**Example.** Wurtzite ZnO without Hubbard U (VASP-Flow would otherwise put its tabulated U on Zn, an oxide), `LASPH` in every VASP step, stricter electronic convergence only in the supercell runs, a $3 \times 3 \times 2$ supercell, 20 snapshots, a denser FOCEX mesh, and your own band path Γ–M–K–Γ–A, which replaces `kpbs.params`:

```
Project: ZnO
RANKS: 1 1 1 0 0 0 0 0
FUNCTIONAL: PBEsol
GGA_U: OFF
MPI: 40
SUPERCELL: 3 0 0  0 3 0  0 0 2       # rows: supercell vectors in primitive units
SNAPSHOTS: 20
FOCEX_MESH: 12 12 8
KPATH: 1                              # the band path is given below

INCAR                                 # every VASP step (merged tag by tag)
  LASPH = .TRUE.
INCAR_END
INCAR_snaps                           # the supercell runs only
  NELM = 200
  ALGO = All
INCAR_snaps_END
kpbs_params                           # replaces kpbs.params (FOCEX's band path)
1        # coordinates in units of the primitive reciprocal vectors
60       # k-points per segment
4        # number of segments
G   0.000000 0.000000 0.000000
M   0.500000 0.000000 0.000000
K   0.333333 0.333333 0.000000
G   0.000000 0.000000 0.000000
A   0.000000 0.000000 0.500000
kpbs_params_END
```

After the set-up you can read `01_relax/INCAR` before anything runs.

## 8 Born charges and the dielectric constant

In an insulator, the long-range Coulomb interaction between the displaced ions splits the longitudinal and transverse optical modes near Γ (LO–TO splitting). FOCEX includes it with the Born effective charges $Z^*$ and the high-frequency dielectric constant $\varepsilon_\infty$; without them the optical branches near Γ are wrong.

- from_scratch: the agent computes them (task dfpt) when the SCF shows a band gap, writes them to `dielectric.params`, and makes for comparison a FOCEX run without them in `05_focex/NOBORN`. `DFPT: off` skips them; `on` computes them for any material.
- extract_only: give them in `dielectric.params` in the data folder.
- your own values, in either mode: a `dielectric_params` block (it replaces the file).

**Note.** VASP prints $Z^*_{ij}$ with $i$ the direction of the field and $j$ that of the displacement; FOCEX expects the transpose, which the agent writes. `BORN_TRANSPOSE: no` writes them as VASP prints them; you should not need it.

## 9 The project folder and the results

Figure 1 shows the folders the agent makes; the table lists what each holds.

| file or folder | what it is |
|---|---|
| `focex-agent-instructions.txt` | the copy of your instruction file that the runs read |
| `report.log` | one entry per task: OK or FAILED, minutes, what was checked – read this first |
| `state.json` | the finished tasks and the numbers passed between them |
| `run_workflow.sh` | the job; `resubmit_failed.sh` reruns the failed supercell runs |
| `POSCAR.input, POSCAR.init, POSCAR0` | your structure; reduced and oriented; relaxed |
| `01_relax, 02_scf` | VASP: relaxation, SCF |
| `03_snaps` | SC-SNAPS input and the snapshots `poscar_NNN` |
| `04_supercells` | `snap_NNN/` (one VASP run each), `POSCAR1`, `FORCEDISP1` |
| `05_focex` | FOCEX: inputs, outputs, plots; `NOBORN/` and `LASSO/` when made |
| `06_dfpt` | VASP: Born charges and $\epsilon^\infty$ (`born_charges.txt`), as in VASP-Flow; `02_scf/dfpt` in projects made before 2026-11-07 |
| `07_thermacond` | THERMACOND (`THERMACOND_JOB` 1–3): `kappa_vs_T.dat`, `kappa_wigner.dat`, ... |
| `diagnosis` | written by `diagnose.py`: `report.txt`, `fix_it.sh`, backups |

In `05_focex`: the force constants `fc2.dat`, `fc3.dat`, ... with the cell in `lat_fc.dat`; the dispersion `bs_freq.dat`; the elastic constants and moduli `mech.dat`; thermodynamics and thermal expansion `thermo_QHA.dat`; and the PDFs (`<Name>` is the project folder): `bands-<Name>.pdf`, `freq_bands-<Name>.pdf` (one colour per branch), `bandgrun-<Name>.pdf` (dispersion above the

**The project folder (`focex-agent.py`, mode `from_scratch`)**

folder  written at set-up  written by the runs  made by you  written by the repairs

| | |
|---|---|
| `~/calc/` | where you call `focex-agent.py` |
| `focex-agent-instructions.txt, POSCAR` | your input |
| `Si/` | project folder (`PROJECT_DIR`; default `PROJECT_NAME`) |
| `focex-agent-instructions.txt` | the copy the runs read |
| `POSCAR.input, POSCAR.init, POTCAR` | your structure; reduced and oriented |
| `run_workflow.sh, state.json` | the job (`sbatch` or `bash`); its state |
| `resubmit_failed.sh` | reruns failed supercell runs |
| `report.log, POSCAR0` | one entry per task; the relaxed cell |
| `01_relax/` | relaxation |
| `INCAR KPOINTS POSCAR POTCAR run.sh` | |
| `OUTCAR OSZICAR CONTCAR vasprun.xml` | |
| `02_scf/` | SCF on POSCAR0 |
| `INCAR KPOINTS POSCAR POTCAR; OUTCAR ...` | |
| `03_snaps/` | SC-SNAPS: the supercell and the snapshots |
| `cell.inp supercell.inp snaps.inp` | |
| `poscar_000 ...  poscar_NNN` | snapshots; `poscar_000` undisplaced |
| `snapshots.xyz` | all snapshots, for viewing |
| `added_NNN-MMM/` | when `SNAPSHOTS` is raised |
| `04_supercells/` | the supercell VASP runs |
| `snap_000/ ...  snap_NNN/` | INCAR KPOINTS POSCAR POTCAR; OUTCAR |
| `unused/` | snapshots you set aside |
| `FORCEDISP1, POSCAR1, stress.params` | forces; supercell; stress of `snap_000` |
| `05_focex/` | FOCEX |
| `*.params, POSCAR1, FORCEDISP1` | the five inputs and the data |
| `fc*.dat lat_fc.dat log*.dat` | force constants and the log |
| `bs_freq.dat mech.dat thermo_QHA.dat ...` | phonons, elastic constants, QHA |
| `makeplots.sh, *-Si.pdf` | the plots |
| `NOBORN/` | with Born charges: the fit without them |
| `LASSO/` | few equations per unknown (`LASSO_RATIO`) |
| `06_dfpt/` | Born charges, $\epsilon^{\infty}$ (only with a gap) |
| `INCAR ...  extract_born.py; OUTCAR born_charges.txt BORN` | |
| `07_thermacond/` | THERMACOND (`THERMACOND_JOB` 1–3) |
| `lat_fc.dat fc2.dat fc3.dat *.params` | `kappa_vs_T.dat`, ... |
| `diagnosis/` | `diagnose.py`: report.txt, fix_it.sh |

Figure 1: The folders made from where you call `focex-agent.py` (here `~/calc`, with `PROJECT_NAME: Si`). `focex-agent.py` writes the project folder, `01_relax/`, `run_workflow.sh` and `resubmit_failed.sh` at once; every other step folder is made when its task runs, in the order of the numbers except `06_dfpt`, which runs right after `02_scf` (the numbering of VASP-Flow). Each VASP folder is an ordinary VASP folder. In mode `extract_only` there are no `01`–`03` folders: `04_supercells/` holds your data, rotated into the FOCEX frame, and `05_focex/` the fit.

Grüneisen parameters along the path), `bvel-<Name>.pdf` (the dispersion coloured by group velocity, and the group velocities against frequency), `all-<Name>.pdf` (Grüneisen parameters, density of states, dispersion coloured by the Grüneisen parameters), `qha.pdf` and `thermo.pdf` (thermal expansion, Grüneisen parameter, bulk modulus, thermodynamic functions, free energy against volume), `fcs-<Name>.pdf` and `chi.pdf`. The FOCEX manual describes every output file. The large VASP files (WAVECAR, CHGCAR, CHG, ...) are deleted when no longer needed; the OUTCARs never.

## 10 When a run stops: diagnose.py

```
A=~/Anharmonic-lattice-dynamics/FOCEX/AGENT
python $A/diagnose.py ~/work/ZnO/ZnO     # one or more project folders
less ~/work/ZnO/ZnO/diagnosis/report.txt
bash ~/work/ZnO/ZnO/diagnosis/fix_it.sh
```

`diagnose.py` itself changes nothing. It lists every task as done, FAILED, INTERRUPTED (the job was killed inside it) or not reached, or reports that the project is still running or not yet started. For a failed task it finds the cause from `report.log`, the OUTCARs, `vasp.out`, `focex.out` and the SLURM logs, and writes `diagnosis/report.txt` (what failed, where, the evidence, what the fix will change) and `diagnosis/fix_it.sh`. VASP failures are recognised with the rules of VASP-Flow's diagnosis tool; FOCEX, SC-SNAPS and `read_outcar.x` stops by their messages.

| cause | meaning | what fix_it.sh does |
|---|---|---|
| `timeout` | the job hit its time limit | walltime ×2, resubmit |
| `scf_nelm` | SCF not converged in NELM steps | ALGO = All, NELM = 200 |
| `edddav` | the Davidson eigensolver failed | ALGO = All |
| `zbrent` | the relaxation's line search failed | IBRION = 1 |
| `relax_unconv` | relaxation not converged | resubmit (continues from CONTCAR) |
| `oom` | out of memory | KPAR = 1 |
| `mpi_init`, `not_run`, `interrupted` | MPI did not start; never ran; killed | resubmit |
| `others` | disk quota, symmetry, an unknown stop, ... | held, with the error lines |

New INCAR tags go into the INCARs of the failed runs and into the matching block of the project's instruction file (`INCAR_snaps`, ...), so that later runs use them too. Before it changes anything, `fix_it.sh` checks that nothing is running and that nothing changed since the diagnosis, and copies every file it edits to `diagnosis/backup_<date>/`; then it resubmits (`sbatch` with a SLURM profile, otherwise `bash` in the background). When the fix needs your judgement – a cause it does not know, a fix that was already tried, a new supercell – the script is *held*: it prints what to do and stops, and `bash fix_it.sh -anyway` runs its suggested commands as they are. Run `diagnose.py` again when the resubmitted job has finished.

## 11 Thermal conductivity

The last task, `thermacond`, runs THERMACOND on the force constants of `05_focex`. Whether it runs is decided by the THERMACOND job, line 8 of `latdyn.params`, which you set in the instruction file:

| THERMACOND_JOB | what THERMACOND does |
|---|---|
| 0 (default) | nothing: the task is skipped (FOCEX has already given the bands, DOS and Grüneisen parameters) |
| 1 | three-phonon linewidths of every mode along the band path (lw-selfenergy.dat) |
| 2 | thermal conductivity: RTA and the direct solution of the BTE |
| 3 | thermal conductivity: RTA and the iterative solution of the BTE |

```
RANKS: 1 1 1 0 0 0 0 0        # cubic force constants are needed
THERMACOND_JOB: 2
THERMACOND_MESH: 12 12 12     # latdyn.params line 1 for THERMACOND (default: the
    FOCEX mesh)
THERMACOND_T: 100 900 9       # line 6: Tmin, Tmax (K), number of temperatures
# THERMACOND_SOLVER: 0        # line 9, job 2: 0 conjugate gradient, 1 SVD
```

The task copies lat_fc.dat, fc2.dat, fc3.dat, structure.params, dielectric.params and kpbs.params from 05_focex to 07_thermacond, writes there FOCEX's latdyn.params with lines 1, 6, 8 and 9 from these keywords, and runs tcond-2.2.x (the newest) with the MPI command of the profile; 05_focex is not changed. A latdyn_params block (Section 7) sets all 13 lines for both FOCEX and THERMACOND, and its line 8 is the job unless THERMACOND_JOB is given. The mesh is the expensive choice: the FOCEX default (fine) is usually far too dense, so set THERMACOND_MESH and converge kappa with it.

The report gives, at up to seven of the temperatures, $\kappa_{RTA}$, the full (direct or iterative) solution and the coherences term $\kappa_C$ of the Wigner formulation (W/mK, average of the diagonal), and a warning if the full solution falls below the RTA, which it should not do (the direct solver has an open issue: compare with job 3). The complete results are in 07_thermacond: kappa_vs_T.dat, kappa_wigner.dat, Qvalue.dat (scattering rates) and the other files described in the THERMACOND manual. An existing project gets the task by adding THERMACOND_JOB to its focex-agent-instructions.txt and submitting run_workflow.sh again: the finished tasks are skipped. Changing THERMACOND_JOB, _MESH, _T or _SOLVER later and submitting again runs the task again; V3·$\delta$ is read back from 07_thermacond/V3DELTA when the force constants and the mesh are the same (job 3 after job 2, new temperatures).

## 12 Version history

1.10 2026-11-09: installation in one command, ./configure -install (Section 2, six steps, a table of the problems it reports): on a cluster it finds and loads the modules ~/myenv needs and gnuplot, writes the module load line into ~/.aladyn.env, takes the account from --account, drafts the profile again on a rerun, tests the VASP module and, when it unloads the Python modules, runs VASP through ~/ALADYN/bin/vasp-env; a vasp_std with missing libraries is rejected; make python-tools writes launchers with ~/myenv's Python (no activation); the cores of sinfo (40+) no longer break the profile's JSON; a gnuplot without the pdfcairo terminal counts as no gnuplot (scripts written, the run goes on); VASP_VERSION (from the binary) in ~/.aladyn.env: a warning at the set-up when the functional needs a newer VASP (r2SCAN: VASP 6), and any task stops when its OUTCAR shows that VASP ignored METAGGA; THERMACOND_X is checked at the set-up. Fixed: FOCEX_PROFILE replaced the --profile of the set-up at run time (1.8, 1.9).

1.9 2026-11-09: the serial THERMACOND build (tcond-*-serial.x) is started as one process on threads (TCOND_NUM_THREADS), not with srun, also in aladyn-gui's bundles (one task, -cpus-per-task); without an MPI build the agent takes the serial one; run_workflow.sh

says where its module lines come from (`module spider`); `./configure` writes THERMACOND job examples (MPI, OpenMP) to `~/ALADYN/jobs`.

1.8 2026-11-09: with a SLURM profile VASP runs on the job's tasks (nodes × ntasks_per_node; before, without an `MPI` line, on one core), `MPI: N` below them as `srun -n N`, above them an error; `NODES:`, `NTASKS_PER_NODE:`, `WALLTIME:` reach the job header; `FOCEX_PROFILE`; `./configure` drafts the profile on a cluster; the installation steps follow `./configure`.

1.7 2026-11-09: `MAGMOM`, `ISPIN`, `LDAU` and the other magnetic and +U tags of any INCAR block apply to every step; per-atom `MAGMOM` / `M_CONSTR` carried to the supercell atoms through their sites (antiferromagnets), shown in `04_supercells/snapshots_mag.xyz`; THERMACOND 2.2 plots after the thermacond task.

1.6 2026-11-07: `VASP_RUN: external` and `THERMACOND_RUN: external` (the inputs of a stage are written and the agent stops as WAITING; it goes on from the outputs, which `aladyn-gui` brings back from a cluster); a `thermacond_params` block (the latdyn.params of `07_thermacond`, used as it is); the SCF reads the gap also from an OUTCAR made elsewhere and reuses the relaxation's CHGCAR only when it is there.

1.5 2026-11-07: DFPT in its own folder `06_dfpt`, as in VASP-Flow, and THERMACOND in `07_thermacond` (older projects keep `02_scf/dfpt` and `06_thermacond`); `EFIELD_PEAD` from the gap and the k-mesh, checks of the Born charges and $\varepsilon_\infty$, `born_charges.txt` written (task dfpt). A $Cu_2O$ run had given $\varepsilon_\infty = 207$ and charges summing to $-3.2\,e$: VASP's default field was 2.7 times its own Zener limit.

1.4 2026-11-07: the FOCEX fits hold the invariances exactly (`enforce_inv = 1`; was 0) and the extra LASSO run uses the refit (`enforce_inv = 3`; was 2), as the FOCEX FAQ recommends; `diagnose.py` reports the same version.

1.3 2026-11-06: `focex-configure.sh` finds the programs, the POTCAR library, VASP and MPI and writes `~/.focex-agent.env`; installation and quick start as numbered steps (commands in blue boxes, explanations in amber boxes); the minimal instruction file and a table of every default and how to change it; the manual now says that without `MPI` VASP runs on one core; a diagram of the project folder (Figure 1).

1.2 2026-10-27, with FOCEX 9.0: the undistorted supercell `snap_000` is run with `ISIF = 2`, and `collect` writes its stress, minus the `in kB` line of OUTCAR divided by 10, as `stress.params` (GPa, tensile positive) for FOCEX (Huang rows and elastic constants); the test VASP stand-in prints the stress.

1.1 2026-10-23: task `thermacond`, switched on by `THERMACOND_JOB` (latdyn.params line 8), with `THERMACOND_MESH`, `THERMACOND_T`, `THERMACOND_SOLVER`; a given `TEMPERATURE` is kept; `SUPERCELL` matrix and `PROJECT_DIR` documented; SC-SNAPS periodic images fixed.

1.0 2026-10-23, with FOCEX 8.19: modes from_scratch and extract_only; instruction file `focex-agent-instructions.txt`, `--instr`, `--coord`, `PROJECT_DIR`; `diagnose.py`; this manual.